

PART D — INTEGRATED SERVICE-CENTRE SYSTEM

#THIS IS THE CODE

```
import java.util.Scanner;
```

```
public class Main {
```

```
    // 1. Student Model
```

```
    static class Student {
```

```
        String studentNo;
```

```
        String name;
```

```
        String serviceType;
```

```
        int estimatedServiceTime;
```

```
        public Student(String studentNo, String name, String serviceType, int  
estimatedServiceTime) {
```

```
            this.studentNo = studentNo;
```

```
            this.name = name;
```

```
            this.serviceType = serviceType;
```

```
            this.estimatedServiceTime = estimatedServiceTime;
```

```
        }
```

```
        @Override
```

```
        public String toString() {
```

```
            return String.format("ID: %-10s | Name: %-10s | Service: %-12s | Time: %d mins",
```

```
                studentNo, name, serviceType, estimatedServiceTime);
```

```
        }
```

```
}
```

```
// 2. Queue Structure (The Waiting Line)
```

```
static class Queue {
```

```
    private Student[] data;
```

```
    private int front = 0;
```

```
    private int rear = -1;
```

```
    private int size = 0;
```

```
    private int capacity;
```

```
    public Queue(int capacity) {
```

```
        this.capacity = capacity;
```

```
        this.data = new Student[capacity];
```

```
    }
```

```
    public boolean isFull() { return size == capacity; }
```

```
    public boolean isEmpty() { return size == 0; }
```

```
    public void enqueue(Student student) {
```

```
        if (isFull()) {
```

```
            System.out.println("[Error] Waiting line is full!");
```

```
            return;
```

```
        }
```

```
        rear = (rear + 1) % capacity;
```

```
        data[rear] = student;
```

```
        size++;
```

```
        System.out.println("[Queue] Added " + student.name + " to waiting queue.");  
    }
```

```
public Student dequeue() {  
    if (isEmpty()) {  
        System.out.println("[Error] No students in queue.");  
        return null;  
    }  
    Student removed = data[front];  
    data[front] = null;  
    front = (front + 1) % capacity;  
    size--;  
    return removed;  
}
```

```
public void displayQueue() {  
    if (isEmpty()) {  
        System.out.println("[Queue] Waiting line is empty.");  
        return;  
    }  
    System.out.println("\n--- Current Waiting Queue ---");  
    int idx = front;  
    for (int i = 0; i < size; i++) {  
        System.out.println((i + 1) + ". " + data[idx]);  
        idx = (idx + 1) % capacity;  
    }  
}
```

```
}  
}
```

// 3. Singly Linked List Structure (The service Records)

```
static class Node {  
    Student data;  
    Node next;  
    public Node(Student data) { this.data = data; this.next = null; }  
}  
  
static class SinglyLinkedList {  
    private Node head = null;  
  
    public void insertStudent(Student student) {  
        Node newNode = new Node(student);  
        if (head == null) {  
            head = newNode;  
        } else {  
            Node curr = head;  
            while (curr.next != null) curr = curr.next;  
            curr.next = newNode;  
        }  
        System.out.println("[Records] Saved record for " + student.name);  
    }  
  
    public void displayStudents() {
```

```
if (head == null) {  
    System.out.println("[Records] No records found.");  
    return;  
}  
  
System.out.println("\n--- Student Service Records ---");  
  
Node curr = head;  
  
int count = 1;  
  
while (curr != null) {  
    System.out.println "[" + count++ + "] " + curr.data);  
    curr = curr.next;  
}  
}
```

```
public Student searchStudent(String studentNo) {  
    Node curr = head;  
  
    while (curr != null) {  
        if (curr.data.studentNo.equalsIgnoreCase(studentNo)) return curr.data;  
        curr = curr.next;  
    }  
  
    return null;  
}
```

```
public boolean deleteStudent(String studentNo) {  
    if (head == null) return false;  
  
    if (head.data.studentNo.equalsIgnoreCase(studentNo)) {  
        head = head.next;  
    }  
}
```

```

        return true;
    }
    Node curr = head;
    while (curr.next != null && !curr.next.data.studentNo.equalsIgnoreCase(studentNo))
    {
        curr = curr.next;
    }
    if (curr.next != null) {
        curr.next = curr.next.next;
        return true;
    }
    return false;
}
}

```

// 4. Array Processing Method (Statistics)

```

public static void displayDailyStatistics(int[] serviceTimes) {
    if (serviceTimes.length == 0) return;
    int total = 0, max = serviceTimes[0], min = serviceTimes[0], over10 = 0;
    for (int t : serviceTimes) {
        total += t;
        if (t > max) max = t;
        if (t < min) min = t;
        if (t > 10) over10++;
    }
    double avg = (double) total / serviceTimes.length;
}

```

```

System.out.println("\n=====");
System.out.println("DAILY SERVICE CENTRE STATISTICS");
System.out.println("=====");
System.out.println("Total Students Served    : " + serviceTimes.length);
System.out.println("Total Service Time      : " + total + " mins");
System.out.printf( "Average Service Time      : %.2f mins\n", avg);
System.out.println("Highest Service Time    : " + max + " mins");
System.out.println("Lowest Service Time     : " + min + " mins");
System.out.println("Services Longer Than 10 Mins : " + over10);
System.out.println("=====");
}

```

// 5. Sorting Algorithm Method

```

public static void sortServiceTimes(int[] times) {
    int[] arr = times.clone();
    for (int i = 0; i < arr.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
    System.out.print("\nSorted Service Times (Ascending): ");
}

```

```

for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + (i == arr.length - 1 ? "" : ", "));
}
System.out.println("");
}

```

// 6. Main Interactive Menu

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    Queue queue = new Queue(10);
    SinglyLinkedList records = new SinglyLinkedList();

    int[] dailyServiceTimes = {17, 5, 23, 8, 14, 3, 11, 20, 6, 9};

    while (true) {
        System.out.println("\n=====");

        System.out.println("CAMPUS SERVICE CENTRE");
        System.out.println("=====");
        System.out.println("1. Add student to waiting queue");
        System.out.println("2. Serve next student (remove from queue)");
        System.out.println("3. Display waiting students");
        System.out.println("4. Add student service record (Linked List)");
        System.out.println("5. Display student service records");
        System.out.println("6. Search for student record");
        System.out.println("7. Remove student record");
    }
}

```

```
System.out.println("8. Display daily statistics");
System.out.println("9. Sort service times");
System.out.println("10. Exit");
System.out.print("Select option: ");

int choice;

try {
    choice = Integer.parseInt(scanner.nextLine().trim());
} catch (Exception e) {
    System.out.println("[Error] Enter a valid number.");
    continue;
}

switch (choice) {
    case 1:
        System.out.print("Enter Student No: "); String id = scanner.nextLine();
        System.out.print("Enter Name: "); String name = scanner.nextLine();
        System.out.print("Enter Service Type: "); String type = scanner.nextLine();
        System.out.print("Enter Service Time (min): "); int time =
Integer.parseInt(scanner.nextLine());
        queue.enqueue(new Student(id, name, type, time));
        break;

    case 2:
        Student served = queue.dequeue();
        if (served != null) {
```

```
        System.out.println("[Served] Assisted student: " + served.name);  
        records.insertStudent(served);  
    }  
    break;
```

```
case 3: queue.displayQueue(); break;
```

```
case 4:
```

```
    System.out.print("Enter Student No: "); String rId = scanner.nextLine();  
    System.out.print("Enter Name: "); String rName = scanner.nextLine();  
    System.out.print("Enter Service Type: "); String rType = scanner.nextLine();  
    System.out.print("Enter Service Time (min): "); int rTime =  
Integer.parseInt(scanner.nextLine());  
    records.insertStudent(new Student(rId, rName, rType, rTime));  
    break;
```

```
case 5: records.displayStudents(); break;
```

```
case 6:
```

```
    System.out.print("Enter Student No to Search: "); String sId = scanner.nextLine();  
    Student found = records.searchStudent(sId);  
    if (found != null) System.out.println("[Found] " + found);  
    else System.out.println("[Not Found] Student record does not exist.");  
    break;
```

```
case 7:
```

```
System.out.print("Enter Student No to Delete: "); String dId = scanner.nextLine();  
if (records.deleteStudent(dId)) System.out.println("[Deleted] Record removed.");  
else System.out.println("[Error] Record not found.");  
break;
```

```
case 8: displayDailyStatistics(dailyServiceTimes); break;
```

```
case 9: sortServiceTimes(dailyServiceTimes); break;
```

```
case 10:
```

```
System.out.println("Exiting system. see you next time!");
```

```
scanner.close();
```

```
return;
```

```
default: System.out.println("[Error] Enter a number between 1 and 10.");
```

```
}
```

```
}
```

```
}
```

```
}
```

```
Output

=====
CAMPUS SERVICE CENTRE
=====
1. Add student to waiting queue
2. Serve next student (remove from queue)
3. Display waiting students
4. Add student service record (Linked List)
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Exit
Select option: |
```

so this is the output that shows the index so you hit what you want to do.

```
Select option: 1
Enter Student No: 226141381
Enter Name: Josua Kamati
Enter Service Type: Registration
Enter Service Time (min): 14
[Queue] Added Josua Kamati to waiting queue.
```

#This one show that the student Josua is added to the waiting queue.

```
Select option: 2
[Served] Assisted student: Josua Kamati
[Records] Saved record for Josua Kamati
```

#This one show that the student is assisted and the record for the student is saved.

```
Select option: 5
```

```
--- Student Service Records ---
```

```
[1] ID: 226141381 | Name: Josua Kamati | Service: Registration | Time:  
14 mins
```

#This one still just shows the proof that the student is really saved.

```
Select option: 8
```

```
=====
DAILY SERVICE CENTRE STATISTICS
=====
Total Students Served      : 10
Total Service Time        : 116 mins
Average Service Time      : 11.60 mins
Highest Service Time      : 23 mins
Lowest Service Time       : 3 mins
Services Longer Than 10 Mins : 5
=====
```

#This one shows an overview of the Daily Statistics.

```
Select option: 9
```

```
Sorted Service Times (Ascending): [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
```

#This one shows the calculated stats and sorted array output.